Software Requirements and Design Document

for

CodeConnect — Real-Time Collaborative Code Discussion Platform

Prepared by Abdullah Ismail (24I-0673) Zulqarnain Zulfiqar (24I-0817) Section E

Scope: Software Design and Analysis — Final Project (Deliverables 1–5) **Date:** May 2026

Table of Contents

7. **Class Diagram (DCD)**
8. **Package Diagram**
9. **Deployment Diagram**

---

1. Introduction

1.1 Purpose

This document specifies the requirements and design for **CodeConnect 1.0**, a desktop application that lets developers share code snippets and discuss them in real time inside the same window. It documents the full software engineering artifact set covering Deliverables 1 through 5 of the Software Design and Analysis course: the project proposal, use-case modelling, domain model, system sequence diagrams, and the design class diagram with sequence diagrams.

The audience is the project supervisor, a future maintainer of the codebase, and any team member onboarding to the project.

1.2 Product Scope

CodeConnect is a Java/JavaFX desktop application that combines:

- **A code sharing surface** — users upload, browse, search, bookmark, and view syntax-highlighted snippets.
- **A real-time discussion surface** — every snippet has a discussion room where developers chat over Java sockets. Messages are persisted so the conversation survives restarts.

The product is intentionally bounded: no video/voice, no AI, no mobile client, no cloud hosting. The system runs either on a single PC (SQLite backend) or on a small LAN (MySQL backend with a socket-based event hub).

1.3 Title

CodeConnect — Talk About Your Code, Right Next To Your Code.

A single-window, multi-user desktop tool that puts a code snippet and its discussion room side-by-side, removing the context switch between IDE, chat app, and email when reviewing or debugging code together.

1.4 Objectives

1. **Co-locate code and conversation.** Eliminate the IDE/Slack tab switch by rendering the snippet and its chat in the same JavaFX window.
2. **Real-time collaboration without a server farm.** Use raw Java sockets over the LAN — no message broker, no cloud dependency.
3. **Persist everything that matters.** All accounts, snippets, rooms, and messages are stored in a relational database via JDBC so history is never lost.
4. **Demonstrate sound OOAD practice.** The codebase is the practical embodiment of GRASP patterns (Information Expert, Creator, Pure Fabrication, Controller) and the Deliverable-5 Design Class Diagram.

1.5 Problem Statement

Code review and team debugging conversations today are scattered across three or four tools: the IDE, GitHub, Slack/Teams, and email. The reviewer copies a snippet into a chat window, the chat strips formatting, the discussion drifts away from the code, and three days later nobody remembers which version of the snippet the conversation was about. For a small team or a university lab the overhead is disproportionate.

CodeConnect solves this by making the code the conversation. A snippet is a first-class entity that owns its discussion room. When you open the snippet, you open the chat. When you send a message, every other developer viewing that

snippet sees it instantly via a TCP socket fan-out. When you search, you search code, tags, and discussion in one place.

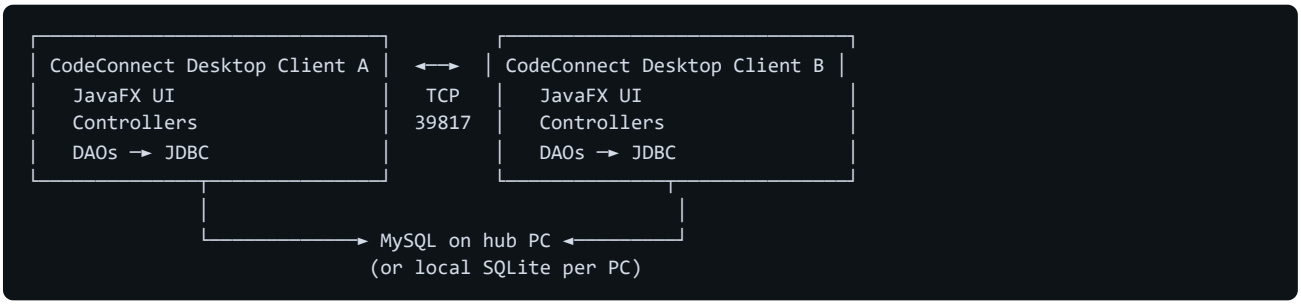
The project is technically feasible because all required components are mature open-source libraries: JavaFX is shipped with OpenJDK, SQLite has a zero-config JDBC driver, and Java's standard `java.net.ServerSocket` is sufficient for LAN-scale broadcast.

---

2. Overall Description

2.1 Product Perspective

CodeConnect is a **new, self-contained product**. It is not a follow-on to an existing product family. It depends only on a Java 21 runtime and a JDBC-compliant database (SQLite by default, MySQL optionally). The only external interface is the local file system (snippet imports, chat-log exports) and, in MySQL mode, the LAN socket.



2.2 Product Functions

At a high level, CodeConnect must let a user:

- **Account management** — register, log in, log out, manage profile, get promoted/disabled by an administrator.
- **Snippet management** — upload (paste or file import), view with syntax highlighting, search by keyword/language/author/date, bookmark, export.
- **Discussion** — open a per-snippet discussion room, send and receive messages in real time, view full history with pagination, mention other users via `@username`.
- **Notifications** — receive a click-through notification when mentioned or when activity happens in a bookmarked snippet.
- **Administration** — administrators can disable users, promote developers to admin, and hide or delete inappropriate snippets.

2.3 List of Use Cases

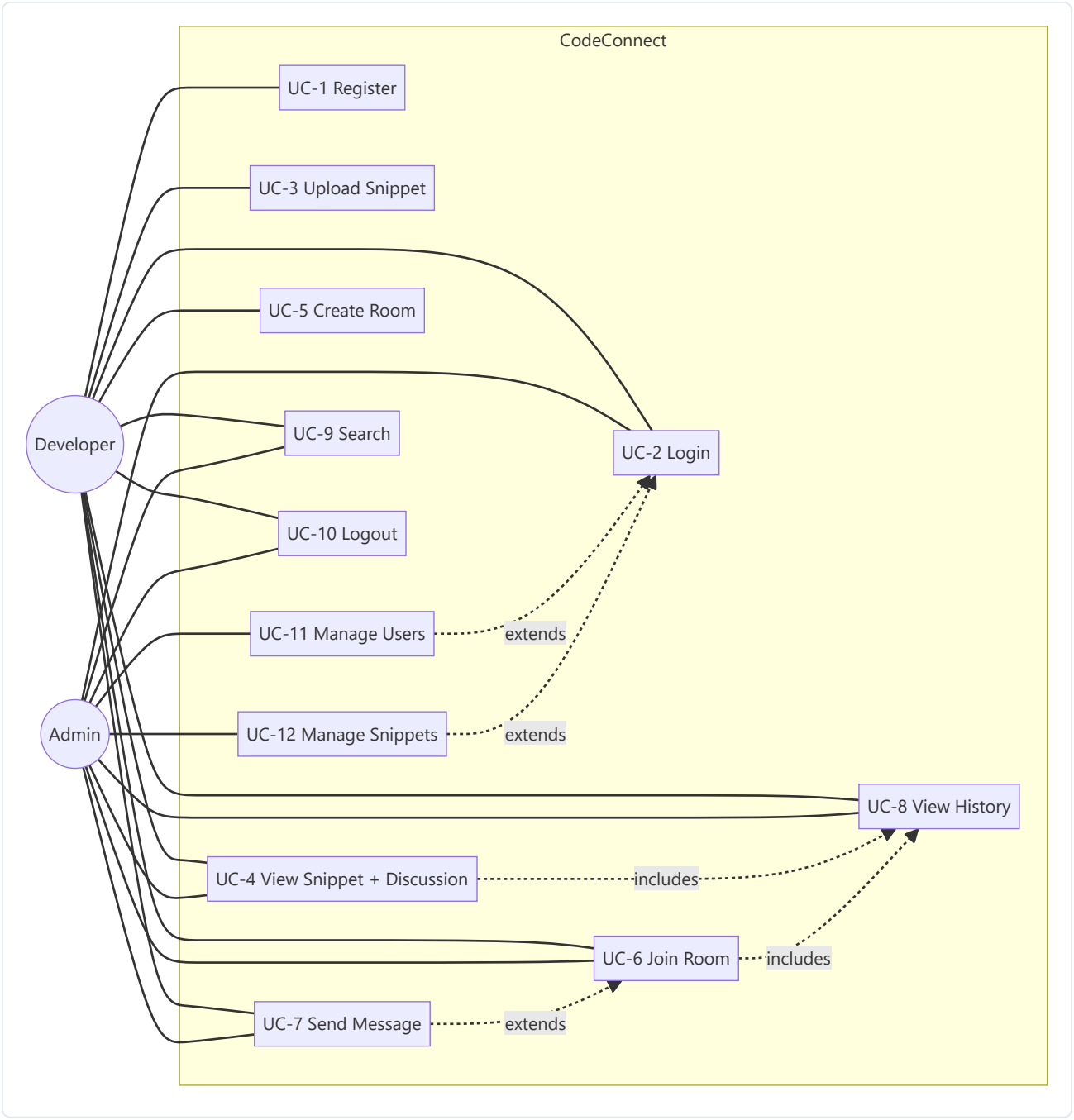
ID	Use Case	Primary Actor
UC-1	Register Account	Developer / Admin
UC-2	Login	Developer / Admin
UC-3	Upload Code Snippet	Developer
UC-4	View Code Snippet (with Discussion)	Developer / Admin
UC-5	Create Discussion Room	Developer
UC-6	Join Existing Discussion	Developer / Admin
UC-7	Send Real-Time Message	Developer / Admin
UC-8	View Message History	Developer / Admin
UC-9	Search Code Discussions	Developer / Admin
UC-10	Logout	Developer / Admin
UC-11	Manage User Accounts	Admin
UC-12	Manage Code Snippets	Admin

2.4 Extended Use Cases

The "extension" rows below are the alternate flows attached to each main use case. They cover both error recovery and security-relevant divergence.

Use Case	Extension	Trigger	System response
UC-1	4a	Username already exists	Reject with "Username is already taken."
UC-1	5a	Database unreachable	Reject with "Database error — could not create account."
UC-2	4a	Wrong password / unknown user	Reject with generic "Invalid username or password." (no enumeration)
UC-2	4b	Account flagged disabled	Reject with "Account is disabled. Please contact an admin."
UC-3	4a	Code body > 256 KB	Reject with "Code is too large (>256 KB)."
UC-3	4b	Required field missing	Reject with field-specific message
UC-4	3a	Snippet was hidden by admin	Show "This snippet is unavailable." for non-admins
UC-5	3a	Room already exists for snippet	Open existing room instead of creating duplicate
UC-6	3a	Room is private and user not a member	Reject with access-denied dialog
UC-7	3a	Empty / >4096-char message	Reject inline; message not persisted nor broadcast
UC-7	5a	Socket bus unreachable	Polling fallback (5 s) keeps the UI converging
UC-9	3a	No matches	Show "No snippets matched your filters." empty state
UC-11	5a	Admin tries to disable themselves	UI guard prevents the action
UC-12	5a	Hide chosen instead of delete	Snippet kept in DB, hidden flag set; admin can unhide

2.5 Use Case Diagram



3. Other Nonfunctional Requirements

3.1 Performance Requirements

Metric	Target	Rationale
Login round-trip on local SQLite	< 250 ms	Sub-second feedback for the most-frequent flow.
Snippet save (upload)	< 500 ms for files up to 256 KB	Covers a single file write to one row.
Real-time message fan-out (LAN, 2–10 clients)	< 100 ms publisher-to-subscriber	LAN round-trip dominated by the socket, not by the DB.
Polling fallback period	5 s	Used only when the socket bus is unreachable.
App cold start	< 4 s on Windows 10/11, 8 GB RAM	Includes JavaFX scene graph and DDL run.
Maximum snippet size	256 KB	Hard cap enforced both client-side and in `SnippetController.MAX_CODE_BYTES`.
Maximum message size	4096 chars	Enforced in `DiscussionController.MAX_MESSAGE_CHARS`.

3.2 Safety Requirements

CodeConnect is not safety-critical (no human/property risk in failure modes), so this section captures **data-integrity safeguards** rather than physical safety.

- **Foreign-key cascades** on `messages.room_id` and on `room_members` prevent orphaned rows when a room is deleted.
- **Soft-hide before hard-delete** for moderation: admins are presented with a "Hide" action before the destructive "Delete" action, reducing the chance of irreversible mistakes.
- **Self-disable guard** in the Admin View prevents an administrator from locking themselves out of the system.
- **Bcrypt hash upgrade on first login** silently re-hashes any legacy plaintext password seeded from older databases.

3.3 Security Requirements

- **Password storage:** all new passwords hashed with BCrypt (work factor 10) via `org.mindrot:jbcrypt`. Plaintext passwords from legacy seeds are upgraded to BCrypt on first successful login.
- **Authentication:** `Account.verifyPassword(String)` is the single authentication primitive (GRASP — Information Expert).
- **No username enumeration:** UC-2 returns the same error message for "user not found" and "wrong password".
- **Privilege-escalation prevention:** the registration form always submits `role = "Developer"` regardless of the request payload. Admin accounts can only be created by another admin via the Admin View.
- **Session management:** the active user is held in `Session` and cleared on logout via `AuthController.requestLogout()` → `Account.logout()`.
- **Disabled accounts:** an administrator can flip the `disabled` flag; disabled accounts are blocked at login.
- **Input validation:** all controllers validate inputs before persistence; size caps are enforced both at controller and DAO level.

3.4 Software Quality Attributes

Attribute	How CodeConnect addresses it
Maintainability	Three-layer architecture (UI → Controller → DAO) with no skip-layer calls; every UI action has an exact controller method per the DCD.
Testability	All controllers accept dependency injection (DAO + SocketServer) via secondary constructors; 19 JUnit tests cover the public contracts.
Portability	Runs on any JDK 21 platform that supports JavaFX. Database backend chosen at start-up via the `DB_KIND` environment variable.
Reliability	A polling fallback layer keeps chats converging even if the socket bus drops; idempotent room joins.
Usability	Single-window layout, keyboard shortcuts on inputs (`Enter` sends), clear error states, role badge + session timer on the sidebar.
Reusability	Controllers and entity GRASP methods are independent of the JavaFX layer and can be re-bound to a CLI or REST front end.
Robustness	Graceful degradation: if MySQL is unreachable, app falls back to SQLite; if socket is unreachable, polling takes over.

3.5 Business Rules

ID	Rule
BR-1	A user must be authenticated to upload, send messages, or open private rooms.
BR-2	Only an Administrator may change another user's role or disable an account.
BR-3	Only an Administrator may hide or delete a snippet authored by a different user.
BR-4	Self-registration always produces a Developer; admin accounts are created by admins.
BR-5	A snippet has at most one discussion room (1:0..1).
BR-6	Messages are immutable except by their author or by an admin.
BR-7	Hidden snippets are invisible to non-admins in all listings and searches.

3.6 Operating Environment

- **OS:** Windows 10 or 11 (primary); Linux/macOS supported on best-effort basis (no Windows-specific APIs are used).
- **Runtime:** OpenJDK 21 + JavaFX 21 (Maven plugin downloads JavaFX SDK during the first build).
- **Database (default):** SQLite 3 via `org.xerial:sqlite-jdbc 3.45+`, file `codeconnect.db` next to the executable.
- **Database (multi-PC):** MySQL 8 via `mysql:mysql-connector-java 8.0.33+`, configured through environment variables.
- **Build tool:** Maven 3.9.6 (bundled in the project under `apache-maven-3.9.6/`).
- **Network:** TCP port 39817 (configurable via `EVENTBUS_PORT`) for the real-time event bus when running multi-PC; port 3306 for MySQL.

3.7 User Interfaces

The UI follows a **dark-monochrome JetBrains-inspired theme** defined in `theme.css`. Every screen uses the same colour palette (`#080C10` background, `#00C896` accent, `#EF4444` danger) and the JetBrains Mono font for code areas.

Common elements on every screen:

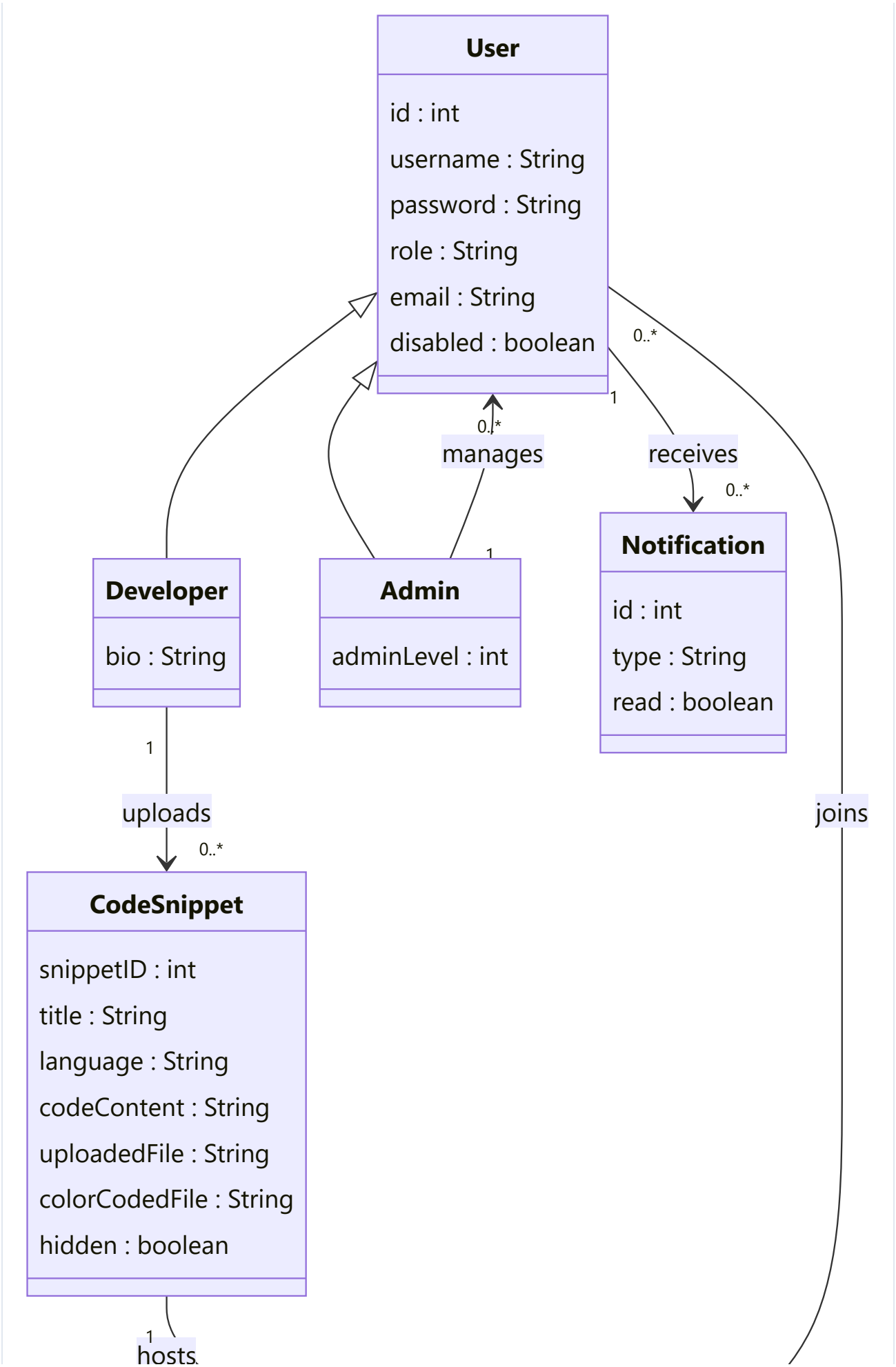
- Top sidebar (62 px collapsed / 240 px expanded) with role badge, snippet count, navigation, session countdown, and logout.
- Right-hand insight panel with active rooms, recent activity, and top snippets.
- Status / error labels in danger red (`#EF4444`); success toasts in accent green.
- Keyboard shortcuts: `Enter` submits forms, `Esc` cancels modal dialogs.

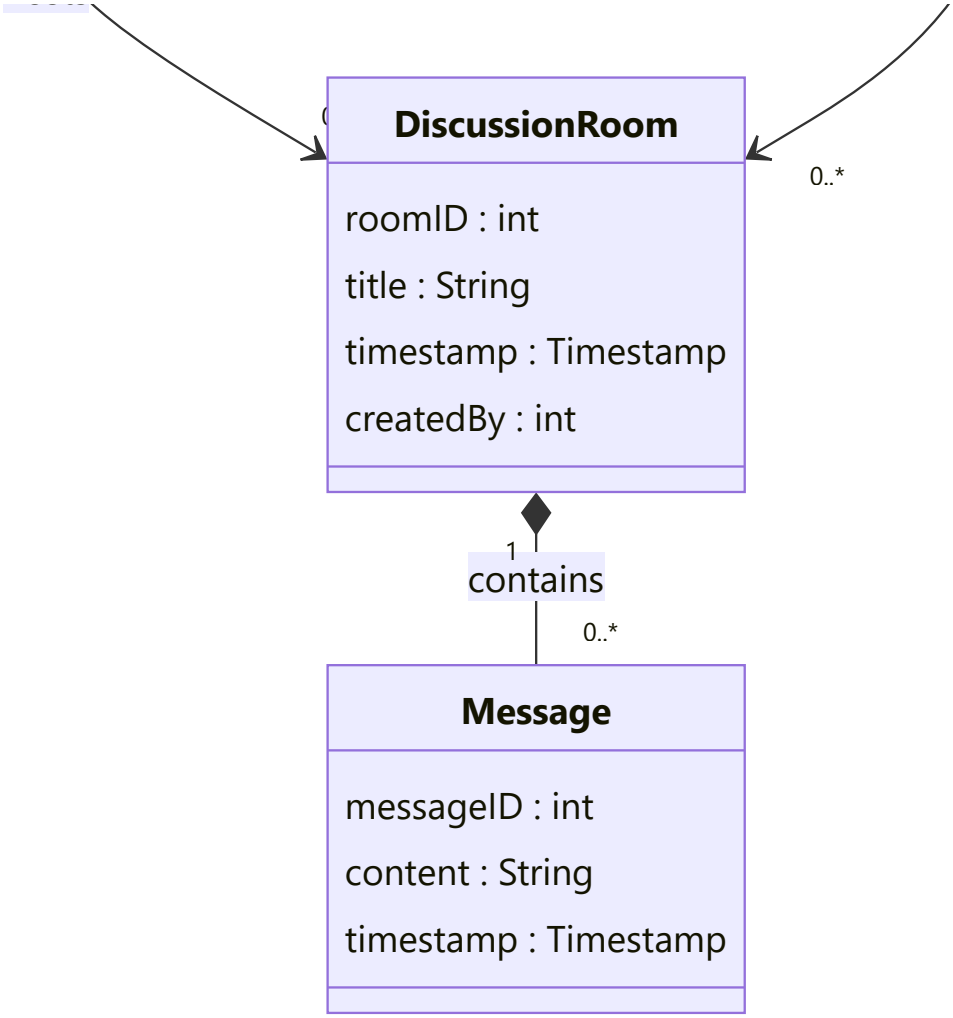
Per-screen highlights:

- **AuthView** — split-hero login/register screen with the wordmark on the left and the form on the right.
- **DashboardView** — KPI tiles + filter bar + grid of snippet cards.
- **DiscussionWindow** — split-pane: code on the left (with syntax highlighting via RichTextFX), terminal-prompt chat on the right.
- **AdminView** — three cards: Users, Snippets, System; Disable/Promote buttons; Hide/Delete on snippet rows.

---

4. Domain Model


User
id : int
username : String
password : String
role : String
email : String
disabled : boolean
0..*
1
0..*
manages
receives
0..*
Notification
id : int
type : String
read : boolean
Developer
bio : String
Admin
adminLevel : int
1
1
uploads
0..*
joins
CodeSnippet
snippetID : int
title : String
language : String
codeContent : String
uploadedFile : String
colorCodedFile : String
hidden : boolean
1
hosts



Multiplicity & relationship summary:

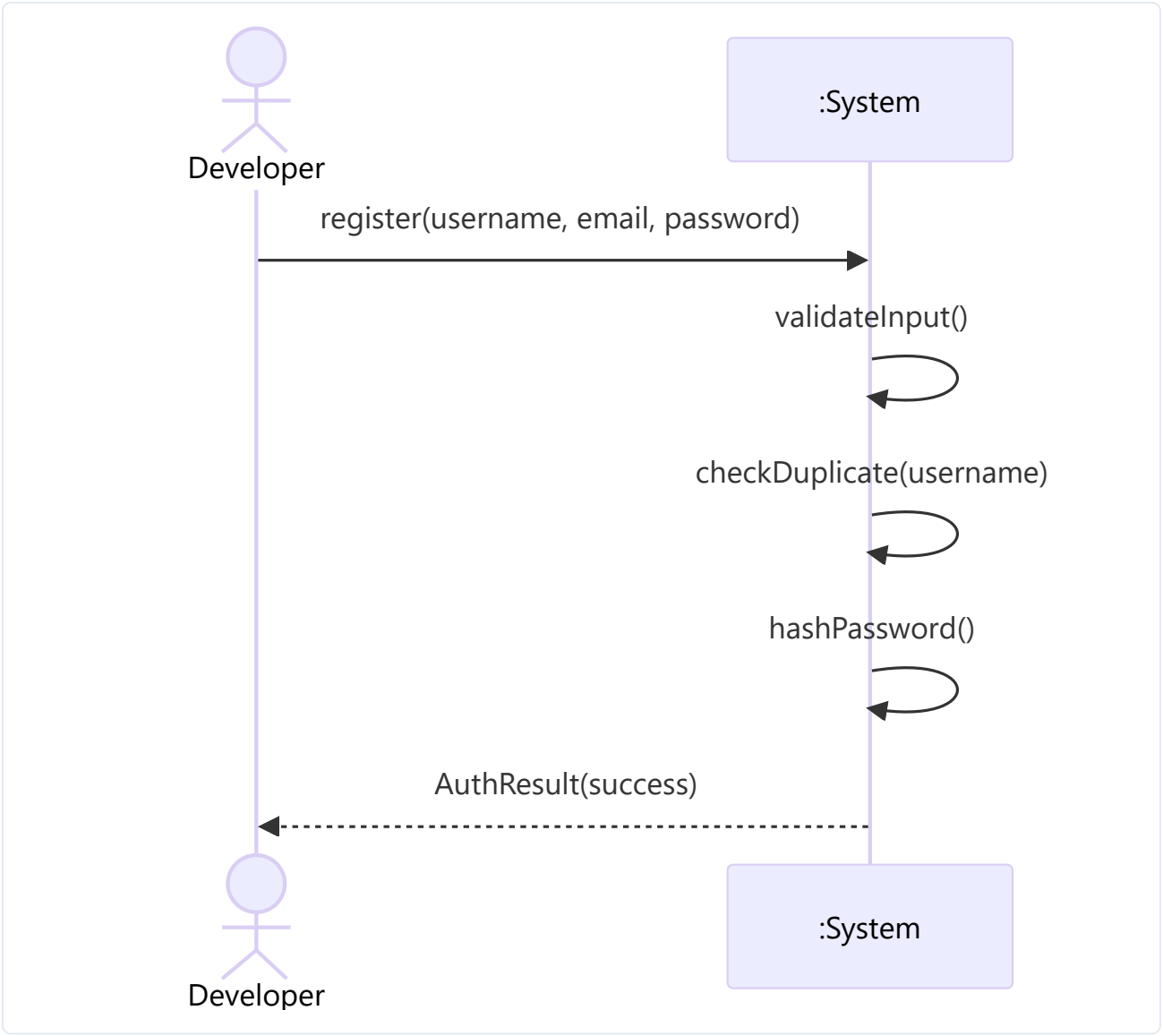
- `User` is the abstract role base; `Developer` and `Admin` are specialisations.
- `Developer` *uploads* zero or more `CodeSnippet`s (1—0..*).
- `CodeSnippet` *hosts* zero or one `DiscussionRoom` (1—0..1, aggregation).
- `DiscussionRoom` *contains* its `Message`s (1—0..*, composition).
- `User` *joins* multiple rooms via the `room_members` junction table.

---

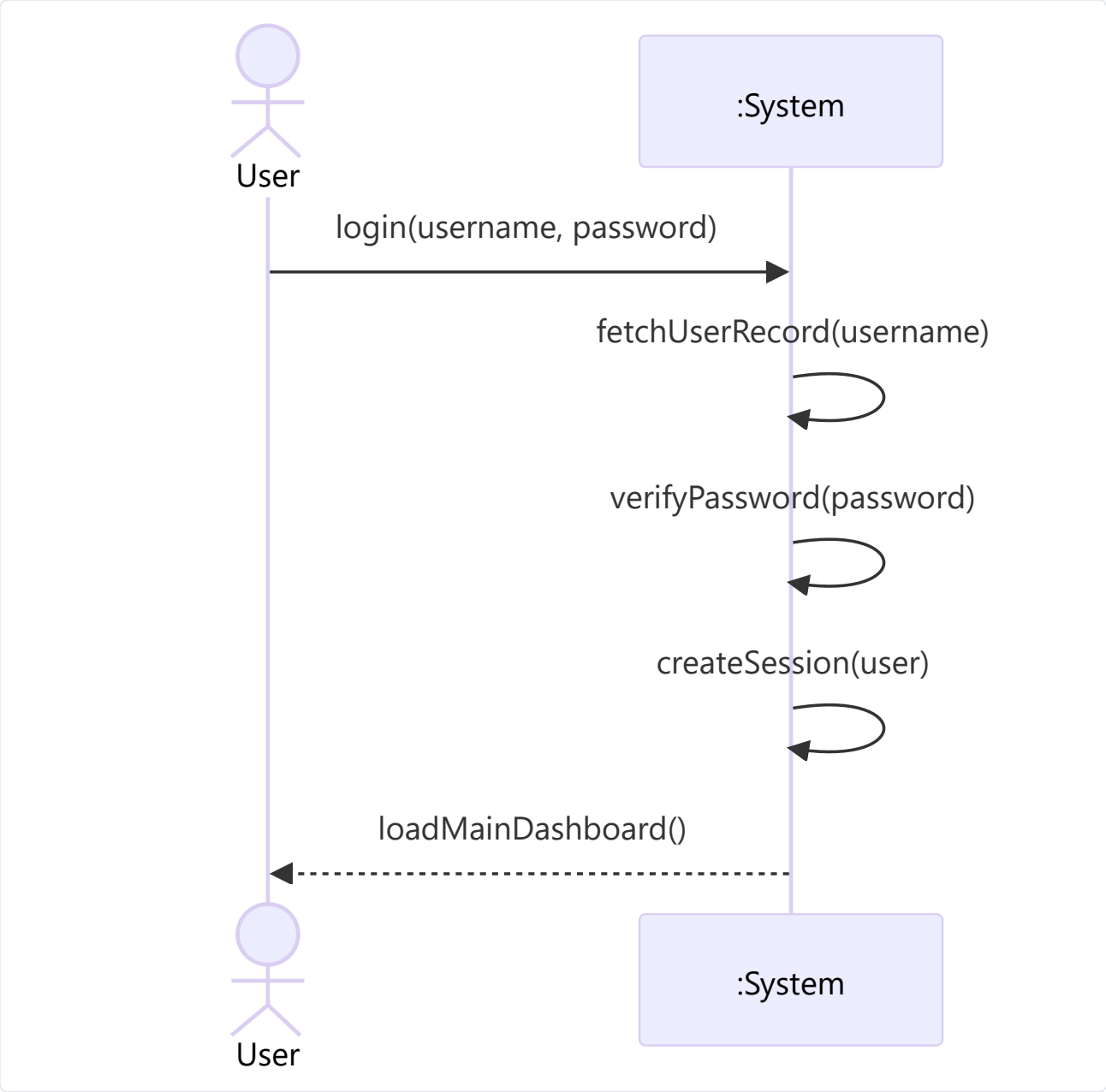
5. System Sequence Diagrams

The 12 SSDs below correspond one-to-one with the 12 use cases. Each SSD's "system event" maps to exactly one method on the Controller layer (see `TRACEABILITY.md` §4 for the full table).

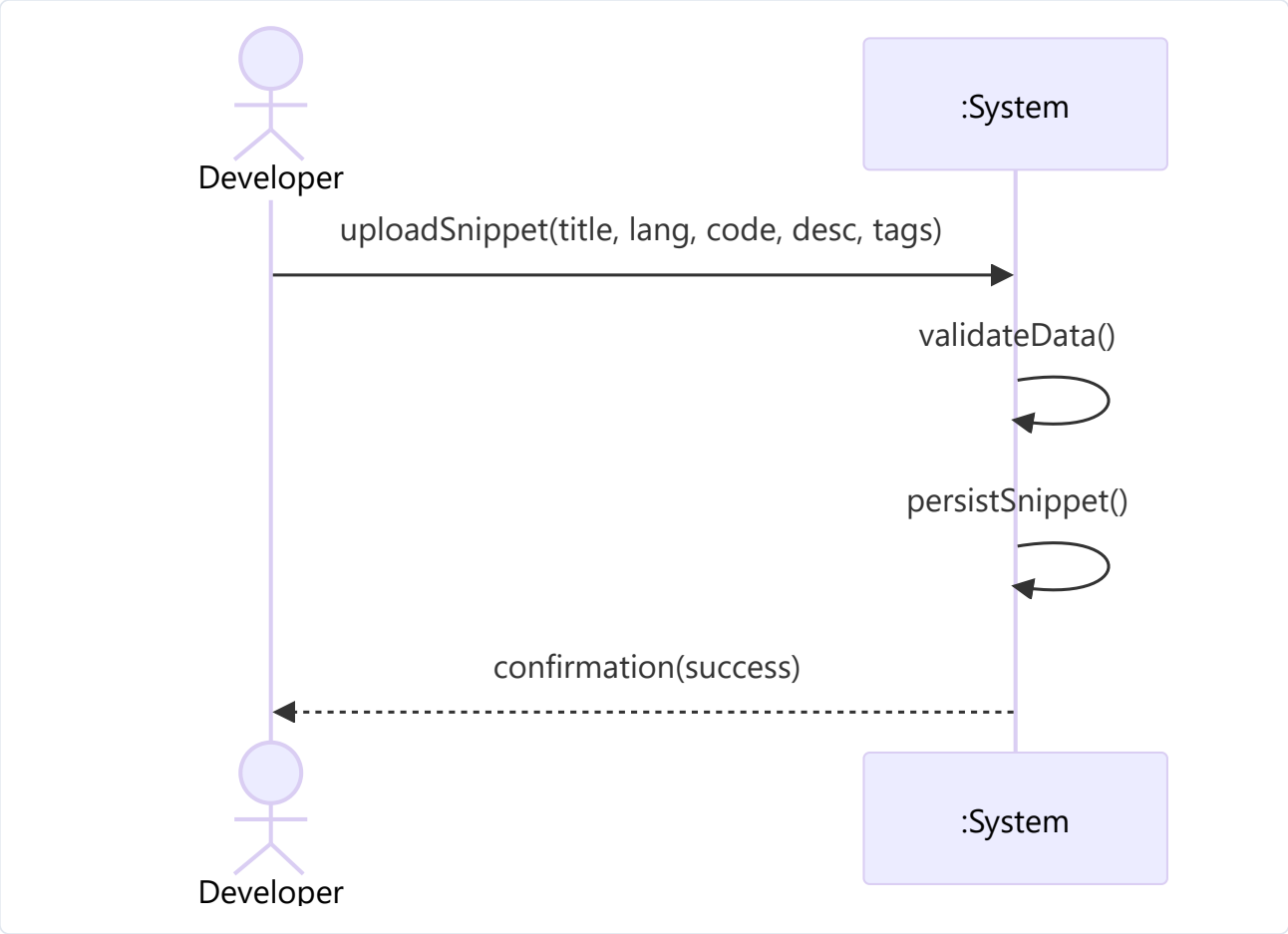
SSD-1 Register Account


Developer
:System
register(username, email, password)
validateInput()
checkDuplicate(username)
hashPassword()
AuthResult(success)
Developer
:System

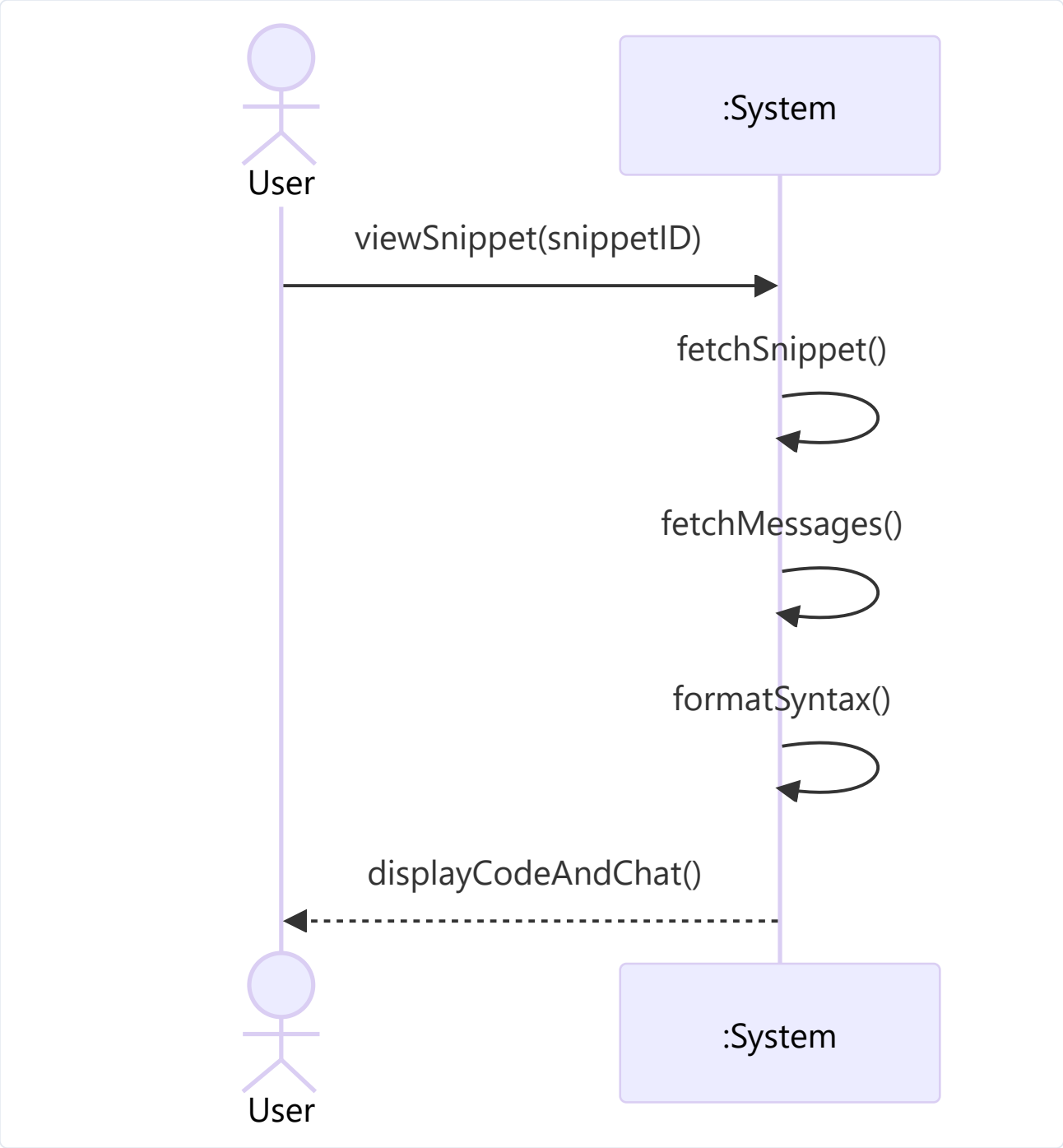
SSD-2 Login



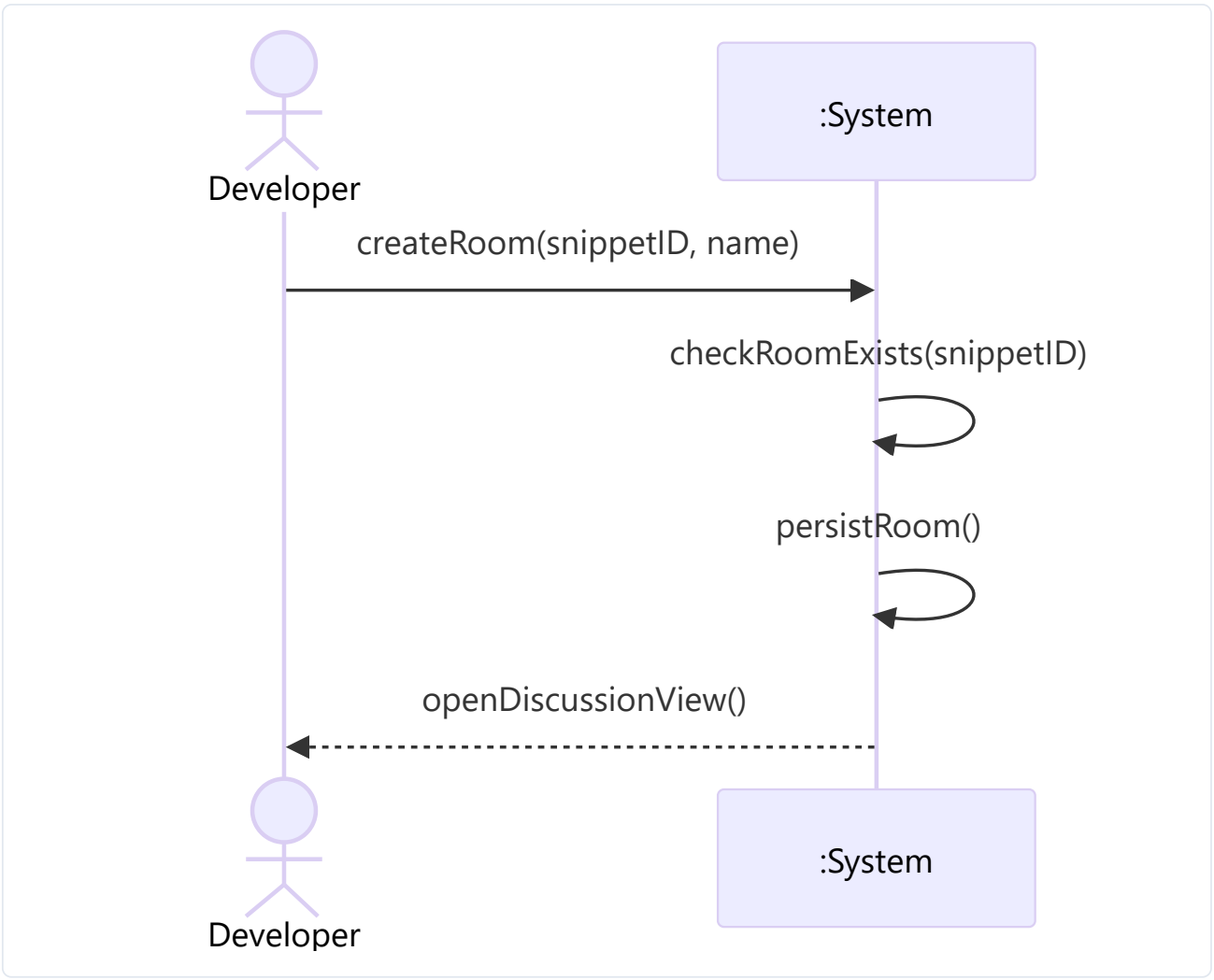
SSD-3 Upload Code Snippet



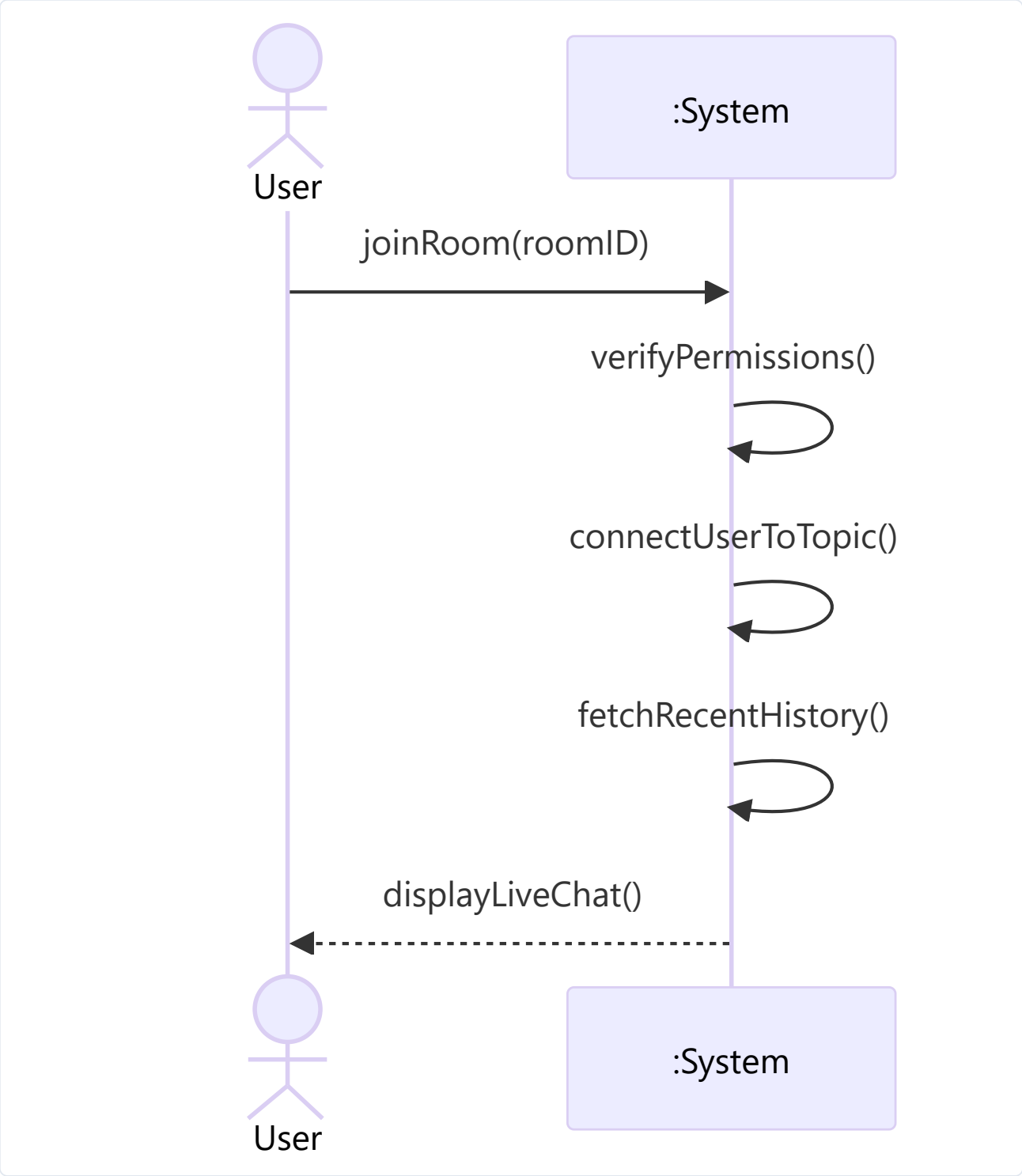
SSD-4 View Code Snippet



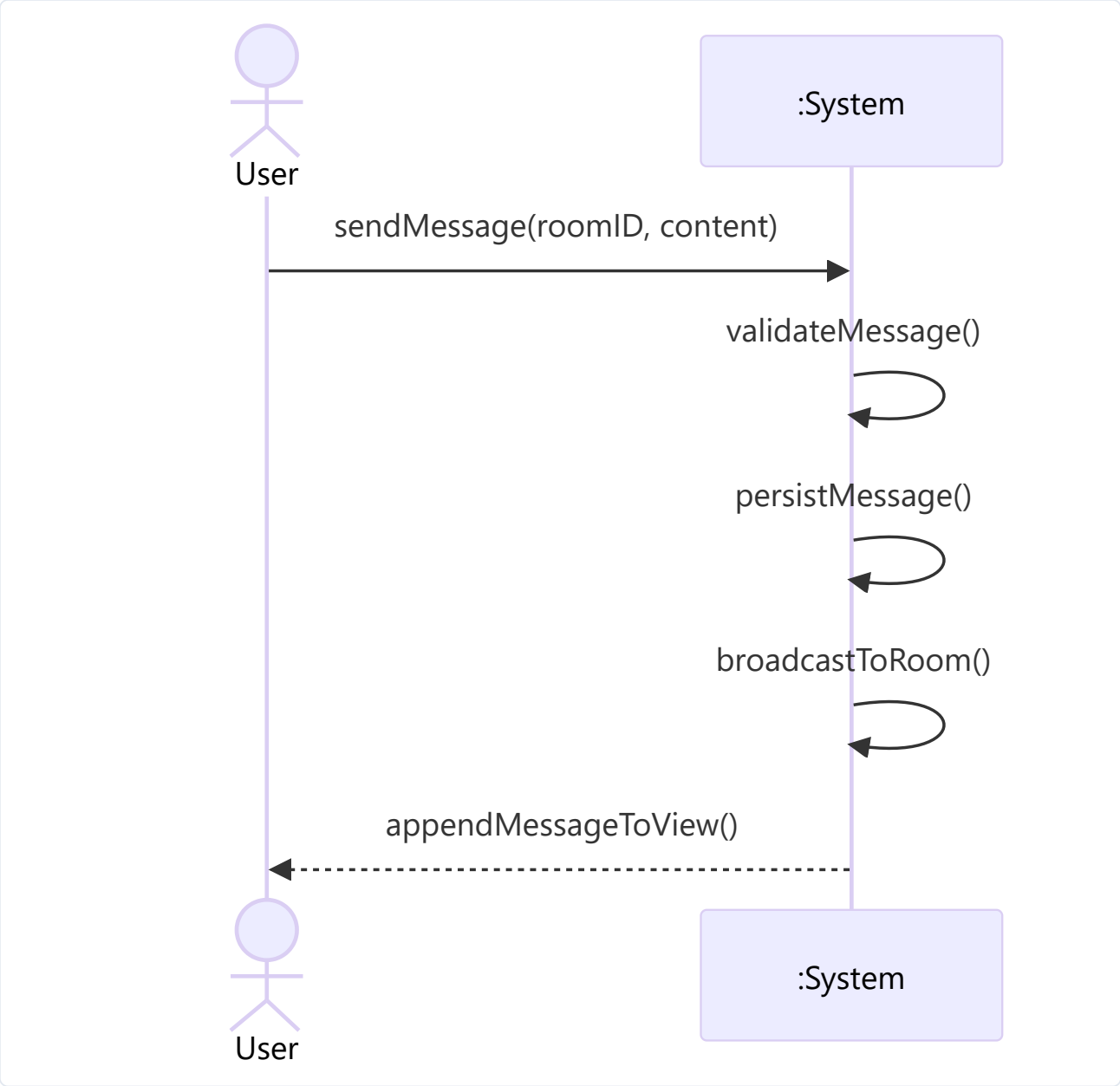
SSD-5 Create Discussion Room



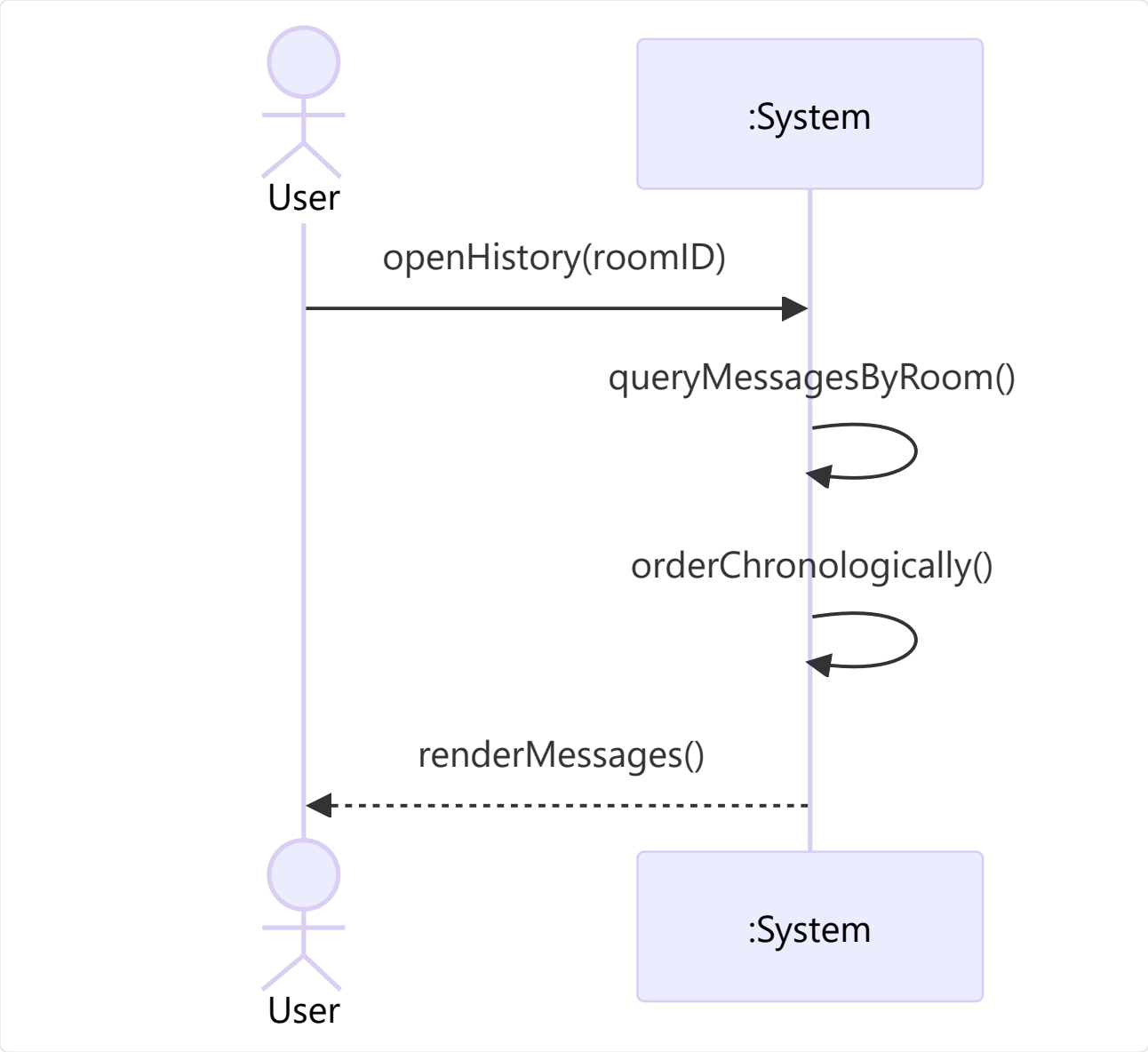
SSD-6 Join Existing Discussion



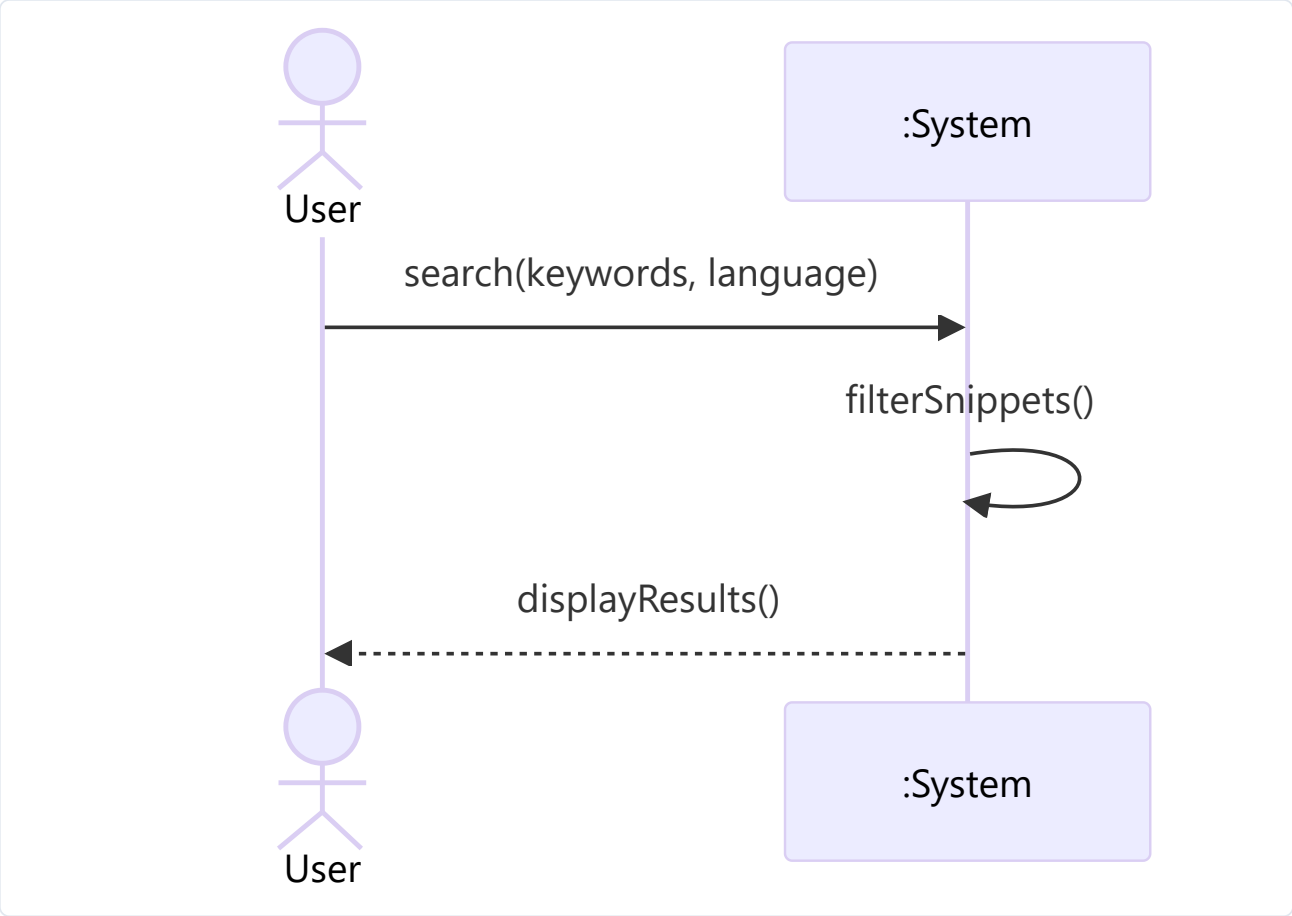
SSD-7 Send Real-Time Message



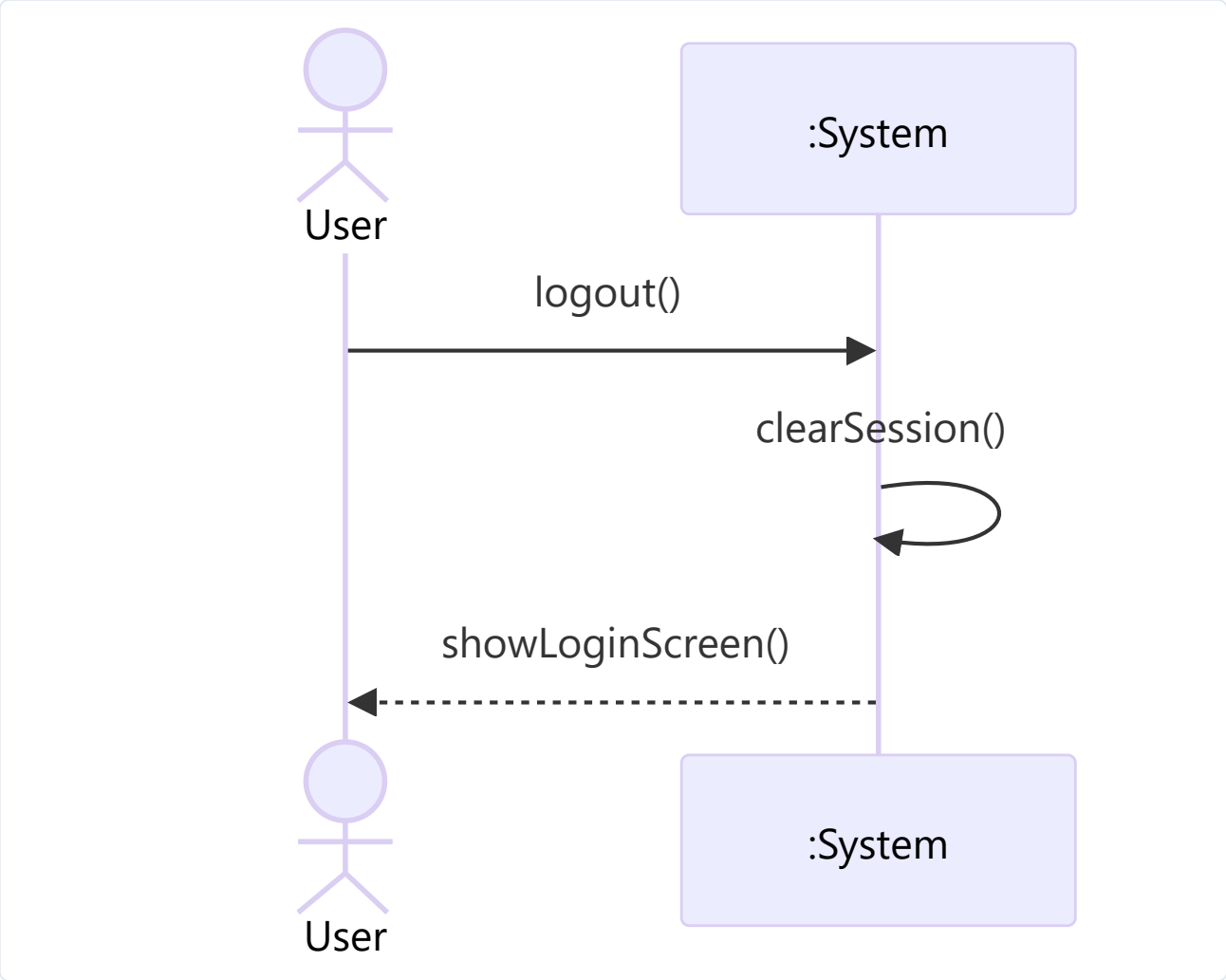
SSD-8 View Message History



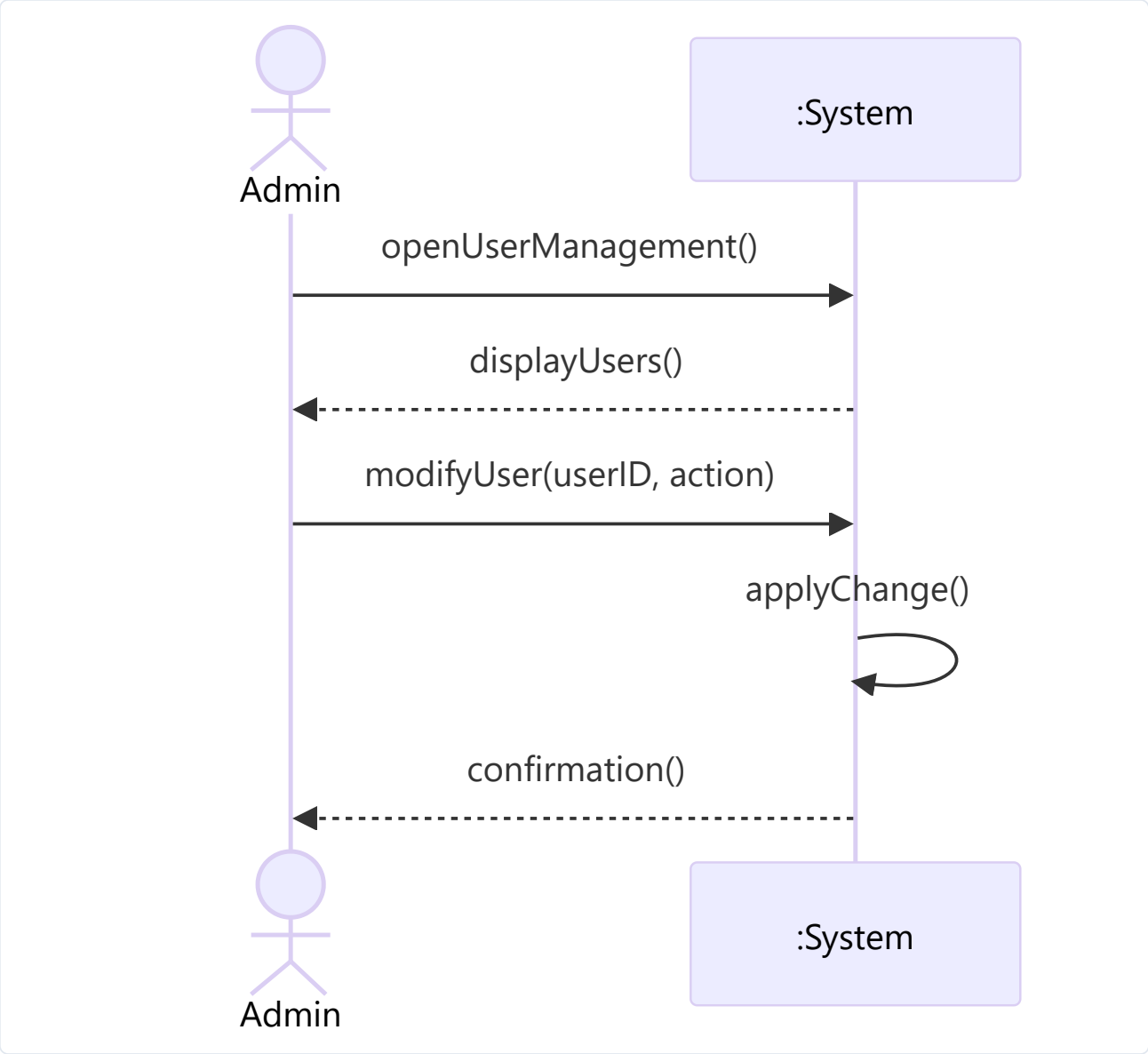
SSD-9 Search Code Discussions



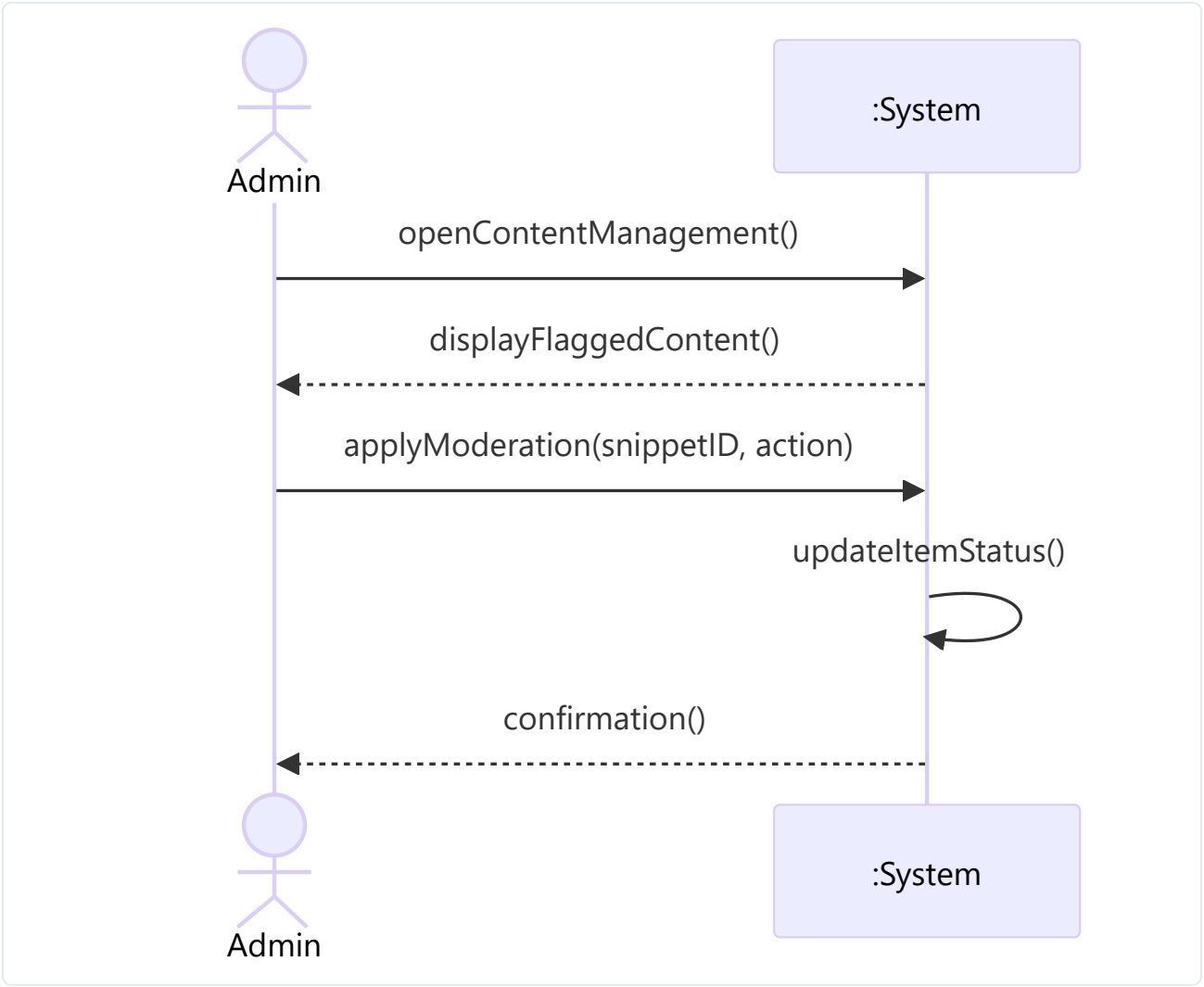
SSD-10 Logout



SSD-11 Manage User Accounts (Admin)



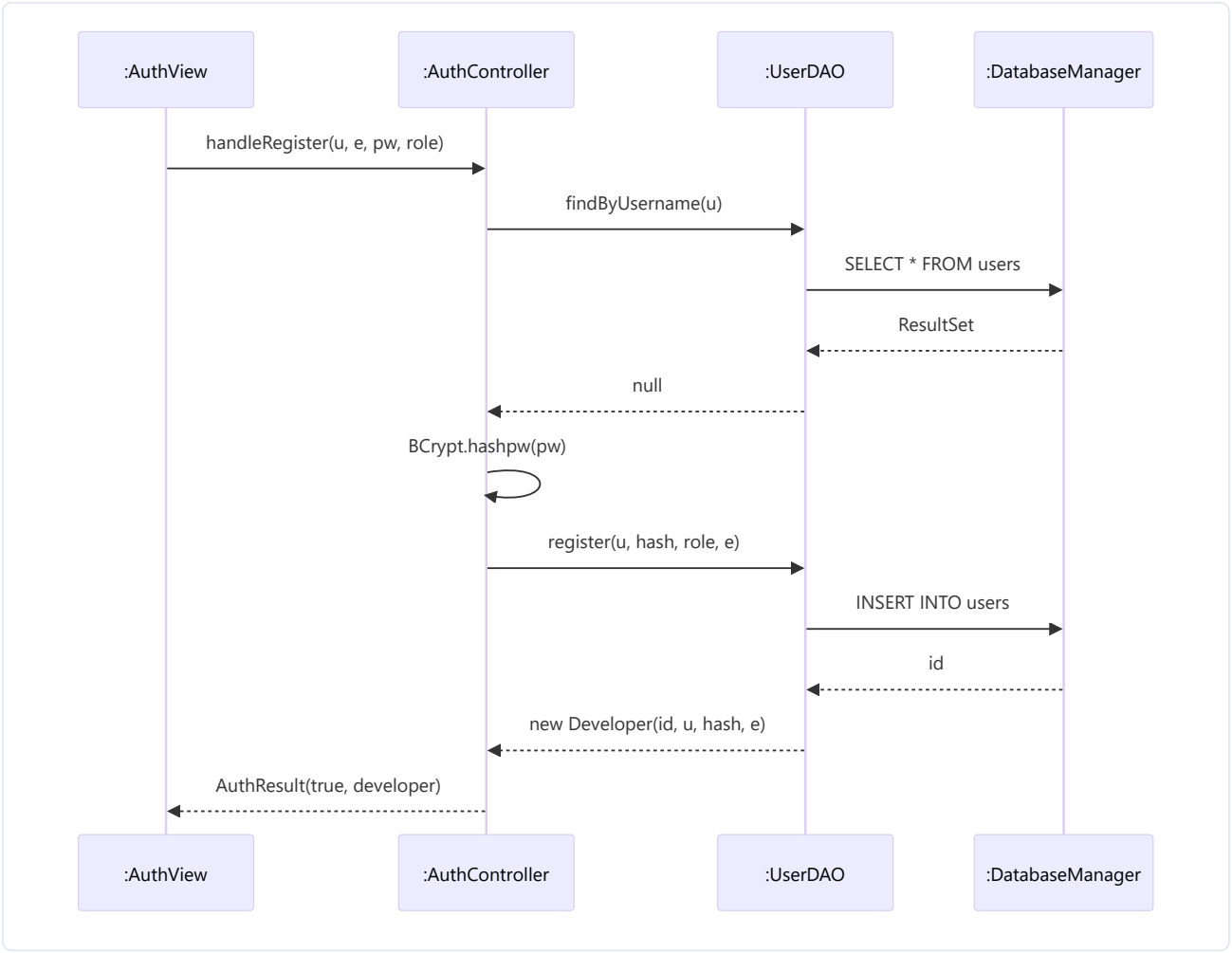
SSD-12 Manage Code Snippets (Admin)



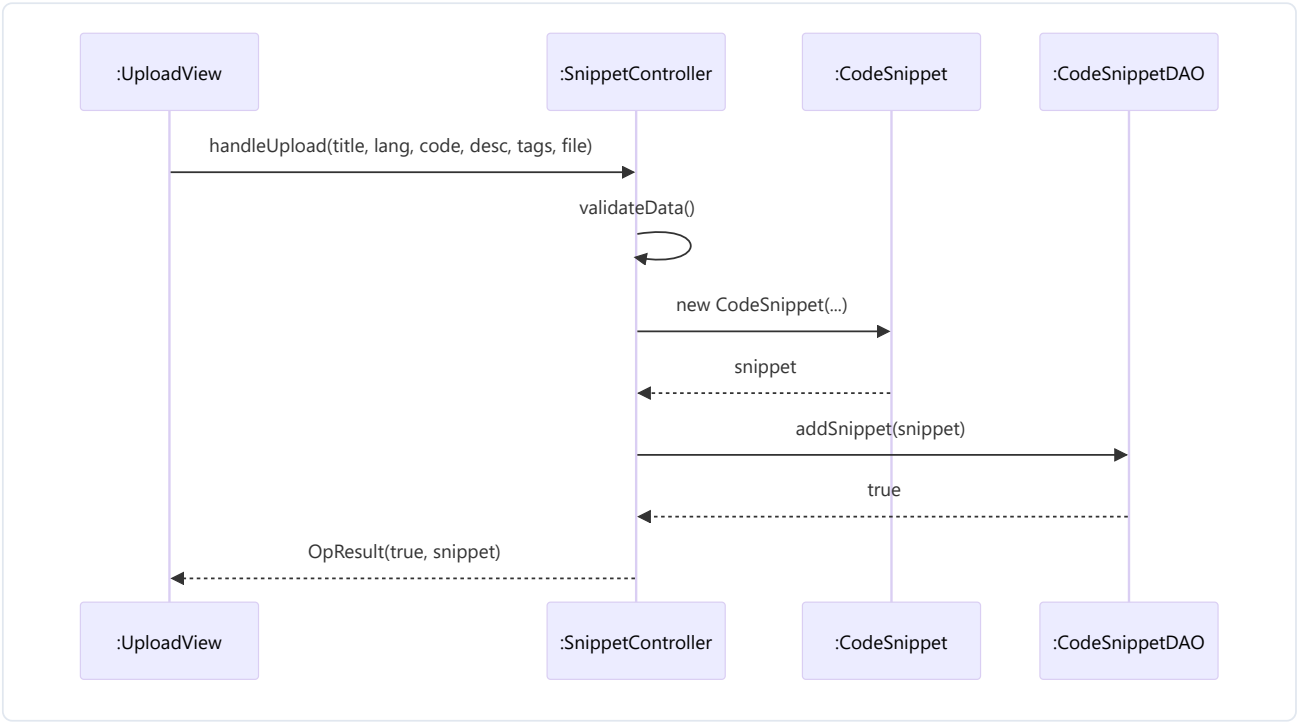
6. Sequence Diagrams

The DCD-level sequence diagrams below show the **internal collaboration** between the UI, Controller, Domain, and DAO layers for the three most illustrative flows: Register, Upload Snippet, and Send Message.

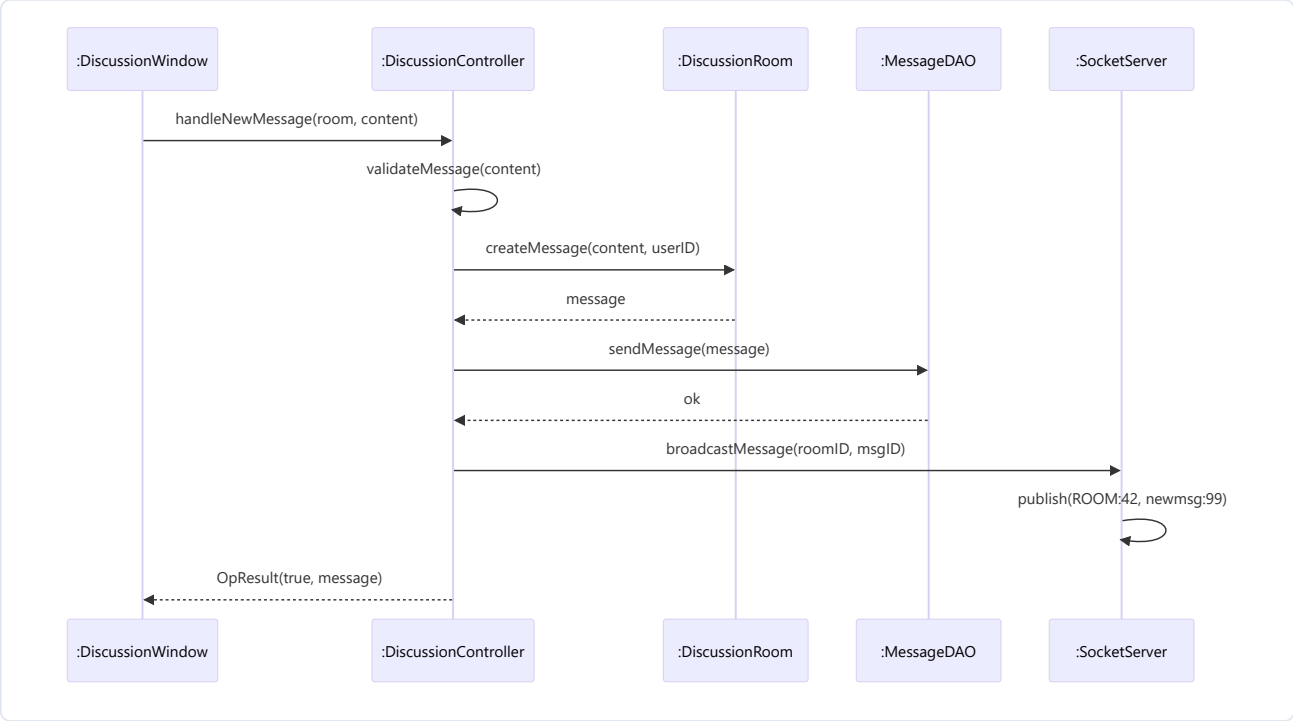
6.1 Register Account (UC-1)



6.2 Upload Snippet (UC-3)



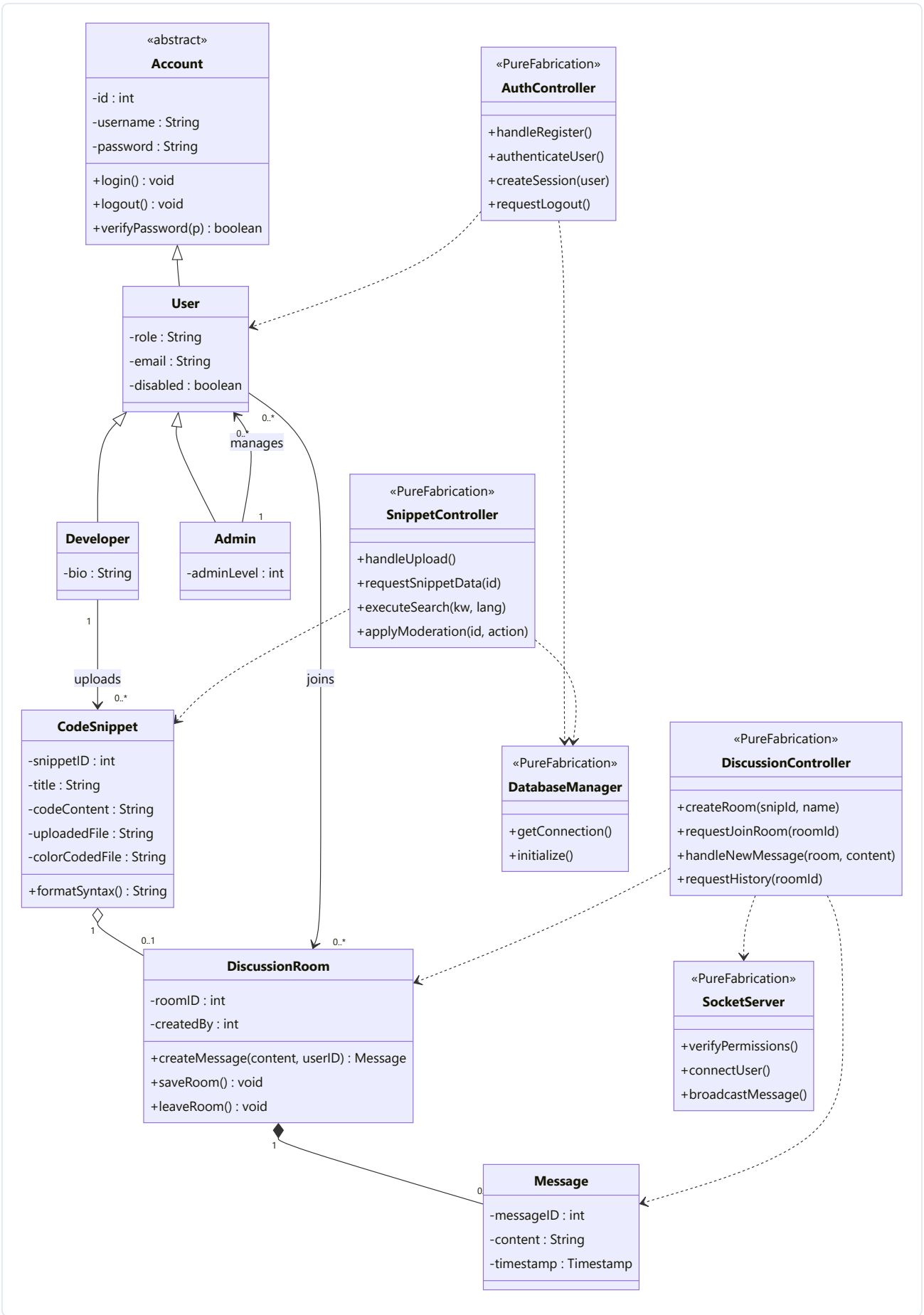
6.3 Send Real-Time Message (UC-7)



---

7. Class Diagram (DCD)

This is the Deliverable-5 Design Class Diagram. Every box maps to a real source file in `src/main/java/com/codeconnect/...`.



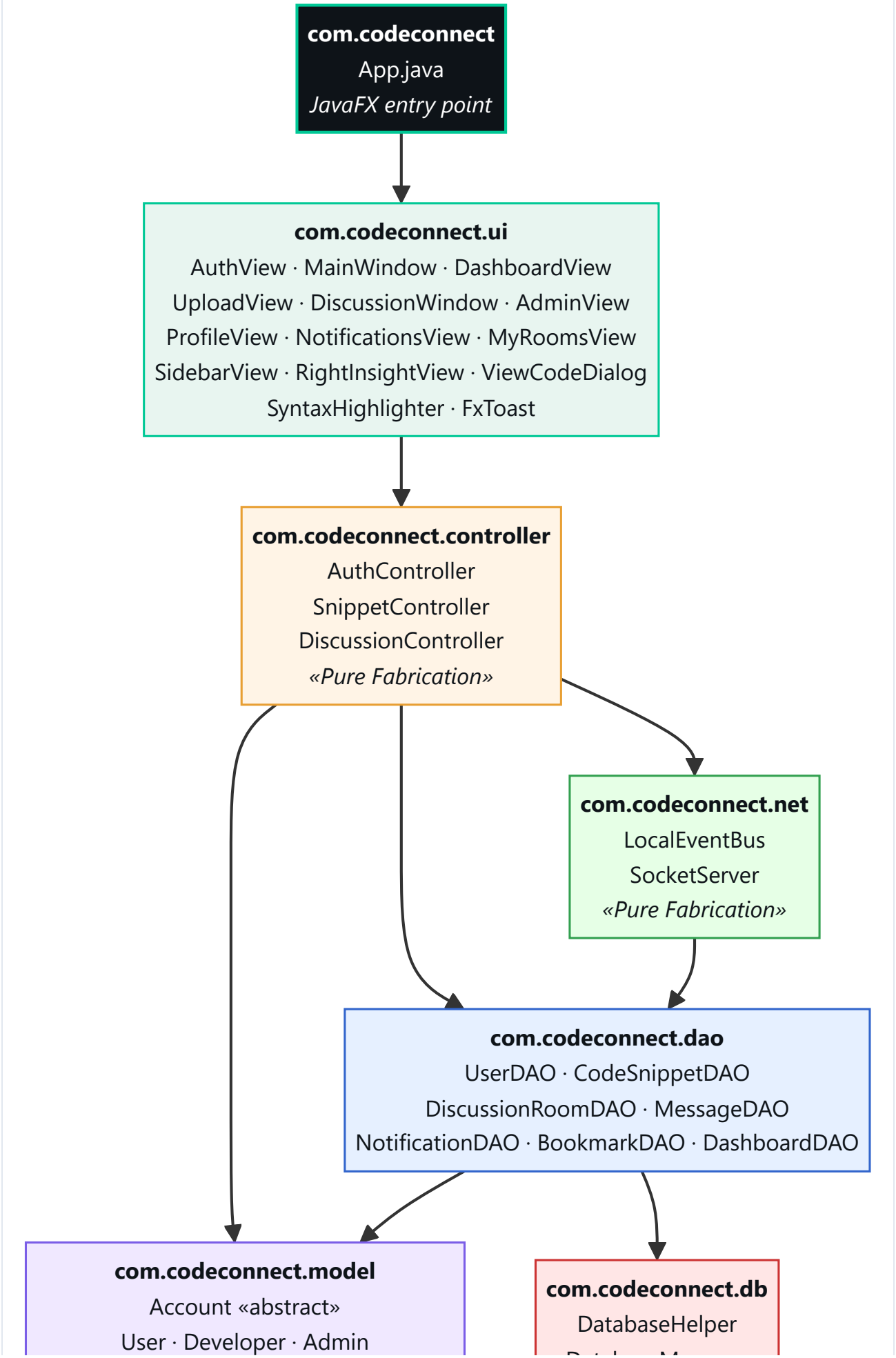
GRASP pattern map:

GRASP Pattern	Where applied
Information Expert	`Account.verifyPassword(...)`, `CodeSnippet.formatSyntax()`
Creator	`DiscussionRoom.createMessage(...)`
Controller	`AuthController`, `SnippetController`, `DiscussionController`
Pure Fabrication	All five `*Controller` / `DatabaseManager` / `SocketServer` classes
Polymorphism	`Account` overridden by `Developer` and `Admin`; `UserDAO.mapRow()` returns the concrete subclass
Low Coupling	UI never imports `java.sql.*`; controllers never import `javafx.*`
High Cohesion	One controller per actor-system event family; one DAO per table family

---

8. Package Diagram

The package diagram below shows the seven packages that make up the CodeConnect codebase and the **strict downward dependencies** between them. The UI layer never reaches across the controller layer; the DAO layer never imports JavaFX.


com.codeconnect
App.java
JavaFX entry point
com.codeconnect.ui
AuthView · MainWindow · DashboardView
UploadView · DiscussionWindow · AdminView
ProfileView · NotificationsView · MyRoomsView
SidebarView · RightInsightView · ViewCodeDialog
SyntaxHighlighter · FxToast
com.codeconnect.controller
AuthController
SnippetController
DiscussionController
«Pure Fabrication»
com.codeconnect.net
LocalEventBus
SocketServer
«Pure Fabrication»
com.codeconnect.dao
UserDAO · CodeSnippetDAO
DiscussionRoomDAO · MessageDAO
NotificationDAO · BookmarkDAO · DashboardDAO
com.codeconnect.model
Account «abstract»
User · Developer · Admin
com.codeconnect.db
DatabaseHelper

CodeSnippet · DiscussionRoom · Message
Session

DatabaseManager
«Pure Fabrication»

Layering rule (enforced by code review and the absence of cross-layer imports):

Layer	May depend on	Must NOT depend on
`ui`	`controller`, `model`	`dao`, `db`, `net`, `java.sql.*`
`controller`	`model`, `dao`, `net`	`ui`, `javafx.*`
`dao`	`model`, `db`	`ui`, `controller`, `net`, `javafx.*`
`db`	(standalone)	everything else
`net`	`dao` (only for resolving subscriptions)	`ui`, `controller`, `javafx.*`
`model`	(standalone)	everything else

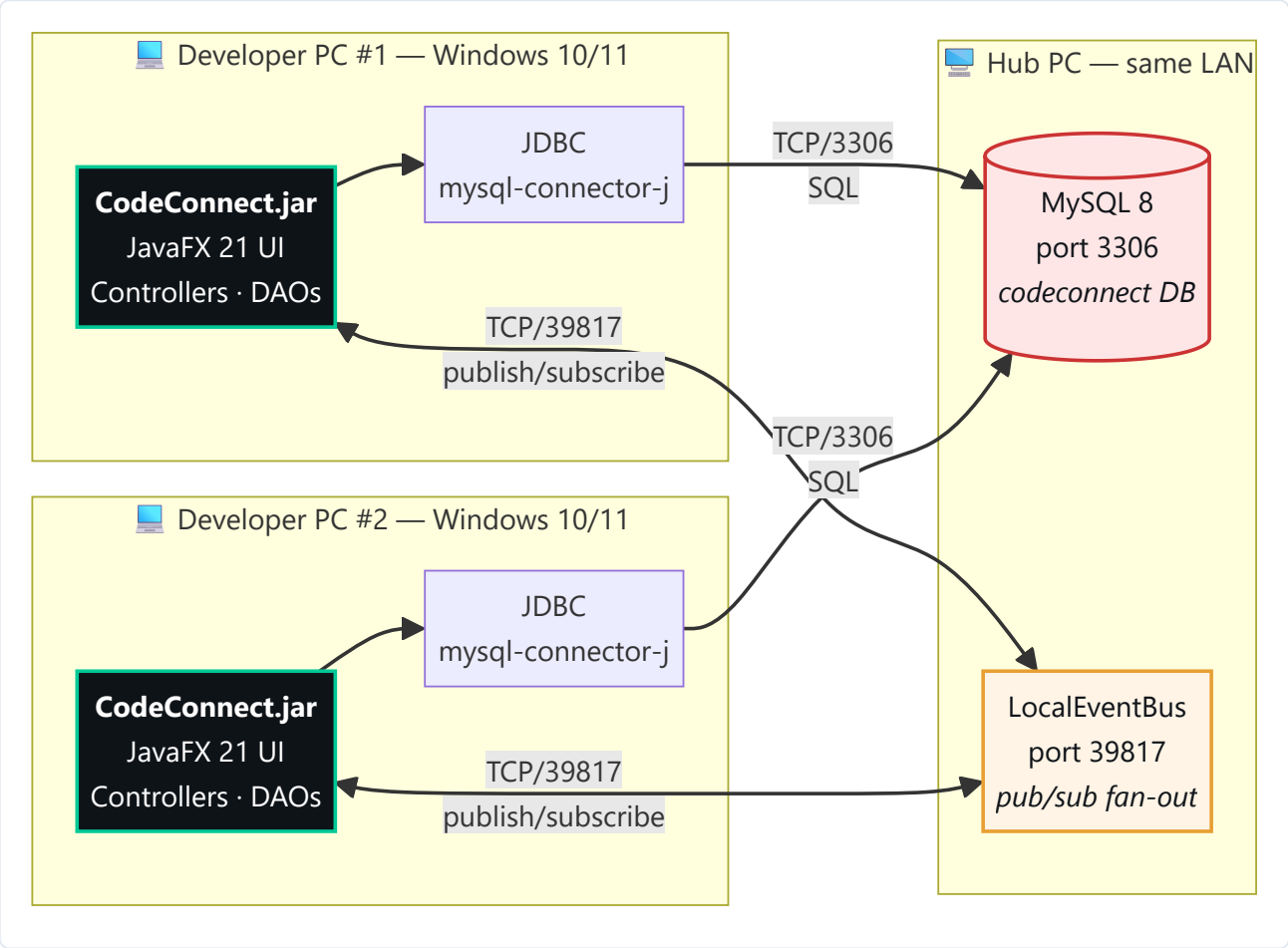
Dependency arrows point downward. There are no cycles.

---

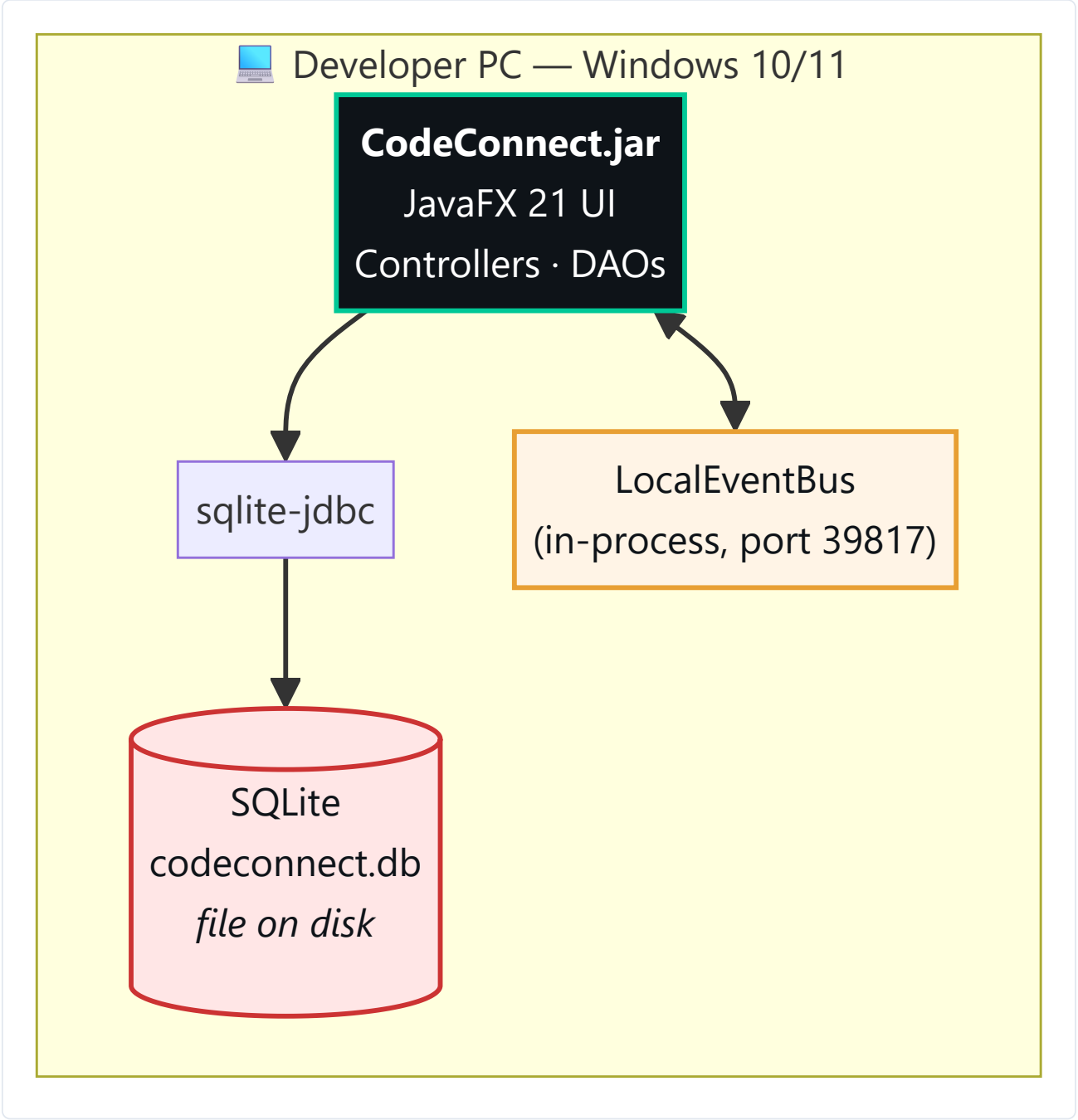
9. Deployment Diagram

CodeConnect supports **two deployment topologies** with the same binary. The runtime topology is selected by environment variables, not by recompilation.

9.1 Multi-PC LAN topology (real-time collaboration)



9.2 Single-PC topology (zero setup)



9.3 Configuration matrix

Variable	Single-PC (default)	Multi-PC client	Hub PC
`DB_KIND`	unset (SQLite)	`mysql`	`mysql`
`DB_URL`	unset	`jdbc:mysql://hub:3306/codeconnect`	`jdbc:mysql://localhost:3306/codeconnect`
`DB_USER`	unset	`codeconnect`	`codeconnect`
`DB_PASSWORD`	unset	*(secret)*	*(secret)*
`EVENTBUS_HOST`	unset (loopback)	hub IP	unset (it *is* the host)
`EVENTBUS_PORT`	unset (default 39817)	`39817`	`39817`

9.4 Hardware & port budget

Resource	Requirement
CPU	Any 64-bit x86 with SSE 4.2 (every modern desktop).
RAM	512 MB free per running client.
Disk	< 50 MB application; < 100 MB DB for typical 6-month usage.
Open ports	3306 (MySQL, hub only), 39817 (event bus, hub only).
OS firewall	Windows Defender inbound rules for 3306 + 39817 on the hub.

The application binaries on every PC are byte-identical; only the environment differs. This means a single zipped distribution file is sufficient for either topology.

Appendix A — Build & Run

```
cd CodeConnect-Submission
.\run.ps1                       # downloads JDK + Maven on first run, then launches
```

To run the test suite (19 JUnit tests):

```
$env:JAVA_HOME = "$PWD\jdk-21.0.2"
$env:PATH      = "$env:JAVA_HOME\bin;$PWD\apache-maven-3.9.6\bin;" + $env:PATH
mvn test
```

Expected: `Tests run: 19, Failures: 0, Errors: 0, Skipped: 0`.

Appendix B — Traceability

`TRACEABILITY.md` (in this folder) gives the line-by-line mapping from every DCD class, every use-case step, and every SSD event to the exact source file and method that implements it.

End of Software Requirements and Design Document.